HDOC DAY-63

Write a C program that reads the size and elements of a variable-length integer array, nums[]. The array contains distinct integers from 0 to size, with exactly one number missing. Pass the array to a function named missing_number() using its address. Use pointers to pass the array between the calling function and missing_number(). The function missing_number() must:

- Find the missing integer from the range 0 to size.
- Display the missing number.
- Assume that all elements in the array are distinct.
- Use the fact that the array contains exactly n numbers from the range 0 to size, with one number missing.
- Display the result as an integer.

Example 1:

Size: 3

Array: 3 0 1

Output:

Missing number: 2

Example 2:

Size: 2

Array: 0 1

Output:

Missing number: 2

Example 3:

Size: 9

Array: 9 6 4 2 3 5 7 0 1

Output:

Missing number: 8

Example 4:

Size: 1

Array: 0

Output:

Missing number: 1

Algorithm:

1. Start.
2. Read the size of the array.
3. Declare a variable-length array nums[size].
4. Read size distinct integers into the array.
5. Pass the array to missing_number() using its address/pointer.
6. Inside missing_number():
    - Calculate the expected sum of integers from 0 to size:
      expected_sum = size × (size + 1) / 2
    - Traverse the array using a pointer and calculate the sum of its elements.
    - Calculate:
      missing number = expected_sum - actual_sum
7. Return the missing number to main().
8. Display the missing number.
9. Stop.

- # C Program:

```
#include <stdio.h>

int missing_number(int *nums, int size)
{
    int expected_sum = size * (size + 1) / 2;
    int actual_sum = 0;

```

```
    for (int i = 0; i < size; i++)
    {
        actual_sum += *(nums + i);
    }

    return expected_sum - actual_sum;
}

int main()
{
    int size;

    printf("Enter size: ");
    scanf("%d", &size);

    int nums[size];

    printf("Enter %d elements: ", size);

    for (int i = 0; i < size; i++)
    {
        scanf("%d", &nums[i]);
    }

    int missing = missing_number(nums, size);

    printf("Missing number: %d\n", missing);

    return 0;
}
```

Test Case Table:

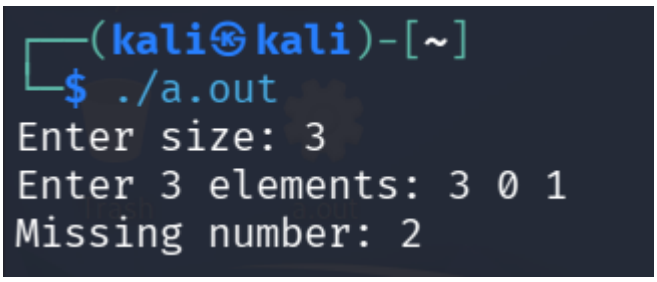
Test Case	Size	Input Array	Expected Output	Actual Output	Result
1	3	3 0 1	Missing Number: 2	Missing Number: 2	Pass
2	2	0 1	Missing Number: 2	Missing Number: 2	Pass

3	9	9 6 4 2 3 5 7 0 1	Missing Number: 8	Missing Number: 8	Pass
4	1	0	Missing Number: 1	Missing Number: 1	Pass
5	5	1 2 3 4 5	Missing number: 0	Missing Number: 0	Pass

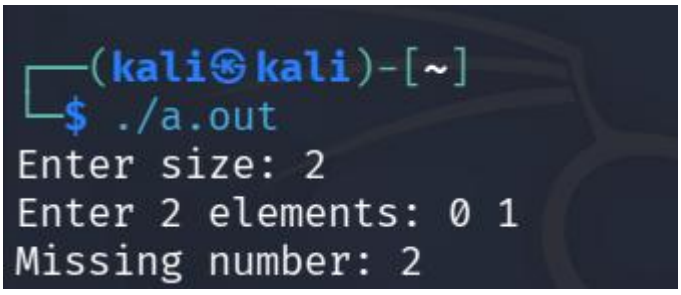
Compilation and Execution:

The following test cases has been compiled and executed successfully.

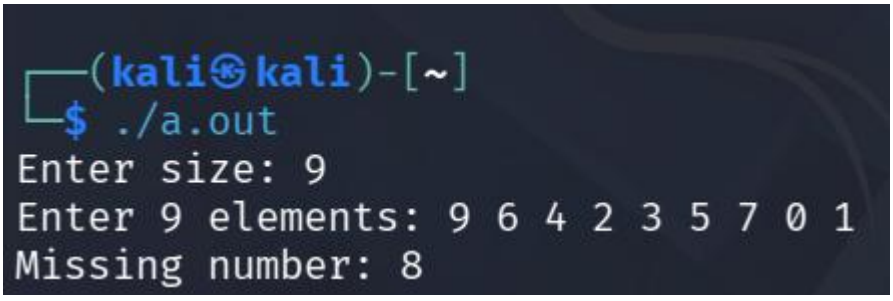
Test Case 1:



Test Case 2:



Test Case 3:



Test Case 4:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size: 1
Enter 1 elements: 0
Missing number: 1
```

Test Case 5:

```
┌──(kali㉿kali)-[~]
└─$ ./a.out
Enter size: 5
Enter 5 elements: 1 2 3 4 5
Missing number: 0
```